Large Synthetic Dataset for Semantic Understanding of Text

Aris Frantzeskakis

Master of Science
Artificial Intelligence
School of Informatics
University of Edinburgh
2025

Abstract

We design and produce a large synthetic dataset of annotations aimed at improving the performance of language models, by directly forcing them to learn certain language features. Through experiments we demonstrate the effectiveness of the data on a small scale. In this process, we show that such a technique could be scaled up to produce a varied corpus of many synthetic datasets, each targeting different kinds of skills.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Aris Frantzeskakis*)

Acknowledgements

Thanks to Dr Wenda Li for supervising this project.

Additional thanks to the teams behind the open-source models used, and to the contributors of the related libraries.

Table of Contents

1 Introduction **1**

1.1 Project Structure . . . 3

1.1.1 Stage 1: Annotation Design . . . 3

1.1.2 Stage 2: Data Generation . . . 3

1.1.3 Stage 3: Data Evaluation . . . 3

1.2 In summary . . . 3

2 Background **4**

3 Synthetic Data Design **6**

3.0.1 Design goals . . . 6

3.0.2 Final design . . . 7

3.0.3 Full examples . . . 10

3.0.4 Observations/Takeaways from Data Design . . . 11

4 Data generation **12**

4.0.1 Fine-tuning phase . . . 12

4.0.2 Fine-tuning observations . . . 13

4.0.3 Generation phase . . . 14

4.0.4 Data quality evaluation . . . 14

5 Benchmark model training and Results **15**

5.1 Model architecture . . . 15

5.2 Model pre-training . . . 15

5.2.1 Data mixtures . . . 15

5.2.2 Data Format and Loss masking . . . 16

5.2.3 Optimizer and hyper-parameters . . . 17

5.3 Fine-tuning and Evaluation . . . 17

5.3.1 Test 1: Blimp Benchmark 17
5.3.2 Test 2: Performance on out-of-domain data 19

6 Conclusions **21**
6.1 Data Generation 21
6.2 Data Effectiveness 22
6.3 Self-Criticisms 22
6.3.1 Small scale Models 22
6.3.2 Data generation outlook 22
6.3.3 Beyond core abilities 23
6.4 Overall Conclusion 24
6.5 Future Work 25

7 Closing notes **26**
7.0.1 Further Particulars in appendix 26
7.0.2 Data & Code 26
7.0.3 Personal outlook 26

Bibliography **27**

A First appendix **31**
A.0.1 On Data-generation phase 31
A.0.2 On evaluation phase 32

Chapter 1

Introduction

Autoregressive LLMs are pre-trained as next-word predictors on large amounts of web scraped text. Through this process, they are expected to learn the underlying patterns of the data and acquire various useful skills, which we can use after they undergo instruction tuning. The next-word prediction task is not directly what we are interested in, but is the only realistic way to gather enough training data for the models to acquire broad "generalist" skills. It is a tractable proxy-task, which by training on, the model can learn useful features for our desired use. Higher quality pre-training data results in better models, leading to the creation of high-quality "filtered" datasets like refined-web[17].

Large synthetic datasets are a promising direction for improving model performance. These were initially somewhat limited, due to needing to be made using conventional programming. However, now that strong LLMs exist, they can be used to produce significantly more diverse synthetic datasets. These datasets can be broadly split into two groups. First, is textbook-examples like the ones used in the Phi[8] models, or huggin-face's Cosmopedia dataset. They consist of high-quality textbook-style extracts, giving a rich description of an individual concept. The second group is task-specific synthetic data, often smaller, that is mainly meant to tune a model to a specific task.

Natural language contains a lot of implicit logic that a reader would understand, but is not explicitly written out. For example, a reader is expected to understand when the word "it" refers to a previously mentioned concept, without the link being explicitly marked. By training models as next-words predictors, we expect them to naturally internalise such logic, as a requirement to better predict the text. While most pre-training datasets, synthetic or not, rely on this process, it is unclear how strict this requirement is or whether it is an efficient way of learning such patterns.

Instead of expecting the model to learn such implicit logic as a by-product, we can also attempt to target it more directly. One method of achieving this, is adding data-annotation tasks to a models pre-training corpus. In such a task the model is given some input, usually a sentence, and is expected to output an annotation that explicitly writes out some of the implicit logic present in the input. A basic example would be marking the part-of-speech of each word. If designed correctly, the model is effectively forced to learn the relevant features and underlying logic, in order to predict the output.

Previous work has shown that using such data can indeed bring benefits. However it comes with two drawbacks. First is that some implementations require task-specific architecture changes, which are difficult to integrate into larger systems. Second and arguably more important is that the relevant data can be difficult to source at scale. It often requires a significant amount of hand-written examples to then train an automated parser to produce a sufficient volume of examples. Additionally, when looking outside a few main corpora the amount of available data drops significantly. There two factors in combination make it difficult to use this technique at scale.

We propose that with the use of modern LLMs, this limitation could be alleviated. In theory, an existing LLM could be efficiently tuned to produce such a dataset in a short time-frame and with minimal effort. If true, small teams could rapidly produce many different datasets each targeting different abilities. These could then be stacked together into a large synthetic corpus that can be used to efficiently train smaller models, or potentially re-enforce certain core abilities in larger models.

Therefore, our two research questions were:

- Can large datasets like this can be created with a small amount of manual effort.
- Is there a benefit in using the result, over standard pre-training data.

For practical reasons we cannot build many datasets just to prove our theory. To that end, in this project we attempt to show a “vertical-slice” of this idea, to show that a full scale version is viable. The limited testing scope poses two challenges for the project. For one, we need to create our data in a relatively task-agnostic way, to show that the process could be repeated for many kinds of tasks. Additionally, we need to make a concrete argument for the data’s usefulness, while testing with a small model.

In this report we outline the work done, note observations during the process that would be useful for reproducing or expanding it, and analyse the results.

1.1 Project Structure

The project spanned 3 stages, each covering approximately 1 month of work.

1.1.1 Stage 1: Annotation Design

In this stage we designed our new annotation schema to be generated. We aimed to target a small subset of skills, mostly syntax adjacent, and have our annotation adequately cover them. Focus was placed on the annotation being information-dense, as to not waste compute when used for downstream training.

1.1.2 Stage 2: Data Generation

This covers the entire process of generating our final dataset. We begin by hand-crafting some examples, with help from an existing reasoning model, to "bootstrap" the process. We then begin fine-tuning an existing LLM (Qwen-3 4B) to teach it to generate the annotation. This is an iterative process in which we cycle between manually correcting the models outputs and fine-tuning a new version. Once the model becomes sufficiently accurate, we use it to automatically generate our final dataset in bulk. If successful, we need to show that the process is reliable, fast and repeatable.

1.1.3 Stage 3: Data Evaluation

Here we evaluate the usefulness of the resulting data. We train 2 small models from scratch, one as a baseline and one including our data. We compare them over a few different tests to measure the down-stream benefit of using the data. This being a small-scale test, it is important to make sure that the data is a flat improvement rather than a trade-off. If the mixed-data model is better in some areas but worse in others, this would suggest that the data simply "shifts the focus" rather than being outright more efficient. While not useless, the value of such a result would be harder to judge.

1.2 In summary

We find that making the data is feasible, but depends on certain factors. Additionally, through a series of tests we show some evidence that, at least on a smaller model, the resulting synthetic data gives a flat performance improvement rather than simply being a trade-off. Overall, the project gave positive results in support of a larger implementation.

Chapter 2

Background

Significant prior work exists, both in the use of syntactic/semantic annotation data to improve LLM performance and in the use of synthetic data created by LLMs. Focusing on the first category, some work has attempted to directly insert additional structured information about the input text into a model, using architecture modifications[25, 18] and/or custom loss functions[21]. For example this can come in the form of additional data bundled together with conventional word embeddings. While this has been shown to work, it comes with a few limitations. Task-specific architecture changes make it difficult to add to a larger model. Additionally, because it relies on existing structured annotations like universal dependencies, it is unclear how it could be scaled further, after a model has sufficiently understood the features captured by the annotation. Lastly, in the cases where the additional information about the input is "embedded" into the model, it is unclear how much the model is required to internalize the understanding of the relevant features. The main difference between these methods and ours, is that we aim to avoid any task-specific changes to the underlying models, to make it feasible to keep stacking different kinds of datasets to target different skills.

Work has also been done with language models trained on data generated using context-free grammars[3, 10, 16]. The aim of these works is broadly to encourage the models to learn helpful biases associated with understanding the general syntactic structure of a given language by first pre-training on large amounts of synthetic text that follows said rules by-construction. The models then continue training on real data or are fine-tuned for specific tasks. Much like the research using modified architectures, it is unclear how this could be expanded into different kinds of annotations.

Closer to our work, are papers[20, 24, 11] that simply add annotation-style tasks into the pretraining data. These require the model to internalize the relevant logic associated

with the annotation, and have no obvious limitations to scaling in terms of the number of different tasks. The main new contribution we want to show in this work, is that the data for various new tasks can be rapidly designed and created using LLMs, instead of relying on pre-existing schemas, parsers or large hand-annotated datasets. A secondary small contribution is to show this process working on auto-regressive models, as many of these past papers focus on sequence-to-sequence architectures.

Significant work also exists relating to the use of LLMs to generate synthetic data. Many examples like self-instruct[22] aim to create small instruction-tuning datasets to tune models on specific tasks. However, this small-scale fine-tuning mostly aims to align a model, not change its underlying features. More relevant to our work are large synthetic datasets like the ones used in the phi series of models[8, 13, 12] or cosmopedia. These consist of many "textbook" examples, which are usually passages of text describing a given concept in detail. These datasets are meant to partially or completely replace the pre-training corpus of a model, increasing its performance due to cleaner data. There are also papers[15] which take real data and paraphrase it into a cleaner format. While these works have the scalability we are looking for, they fundamentally rely on the assumption that the model will learn the implicit patterns present in the text as a by-product of learning to predict it. In this sense they are similar to conventional scraped data.

Arguably closest to our work is a 2024 paper[2], which used an LLM to create a large instruction-style pretraining dataset. A fine-tuned model was used to take passages from a raw text corpus, generate question/instructions relevant to the text, and their corresponding answers. The resulting data was then used to train a new model from scratch. There are 2 main differences we want to focus on for our approach. First, we want to fine-tune a single data generator for each task (here just one, as this is a proof-of-concept) instead of using a joint model. This could potentially allow for cleaner data and for different teams to independently create separate datasets. Second, we aim for a more structured style of annotation, aiming to be more information dense. In contrast, many sample examples in the paper are question-answer tasks, where the answer targets a relatively small part of the input text.

Lastly we want to mention tiny-stories[6]. It showed that pre-training on LLM generated short stories, designed to use limited vocabulary, can allow tiny models to produce coherent English. We used their dataset to train our baseline models during evaluation, as it allowed us to test the idea on a small scale. A bigger test would likely require more time than was available for the project.

Chapter 3

Synthetic Data Design

Each synthetic sample consists of an input (usually a single sentence) and an output, which is some annotation of the input text. The first stage of the project involved designing the annotation schema for the data. The schema had to be rich enough to be good training data while also being easy enough to reliably generate at scale.

3.0.1 Design goals

While the schema is inspired by annotations found in classical NLP research, the priorities of the design were quite different. Most classical annotations are meant to be used by downstream algorithms, often using hand-written logic. In our case the only real goal of the data was to be a valuable training signal for downstream language models. This meant that certain requirements, like having a canonical form for inputs could be significantly relaxed.

3.0.1.1 focuses of the annotation

We choose 3 main things to target with the annotation. First was highlighting co-reference and anaphora. If 2 words are referring to the same "concept" (for example "Tom" and "he"), the annotation needs to point that out. The second was how words link together to form higher concepts. This includes highlighting which adverbs/adjectives "attach" to which words, who the subject/object of an action is etc. We specifically wanted to highlight how different concepts get built up in a sentence. For example in the sentence "I see a blue dog" we want the annotation to first create the concept "blue dog" and then link it to the action "see". The 3rd requirement was a light form of text-grounding. We want the annotation to loosely highlight which parts of the text are

being used to craft different concepts. We considered using exact text spans (using text indices) but choose not to, to avoid facing potential issues with models struggling to count tokens correctly.

3.0.1.2 non-focuses of the annotation

There were some features we were explicitly not interested in. Details like punctuation or filler words, were not a focus, so long as the annotation properly captured the general meaning of a sentence. Additionally, unlike many meaning representations, we did not have a need for a canonical form. This means that the same meaning can sometimes have multiple valid annotations (though in practise this wasn't that common). Lastly we choose to mostly ignore any annotation that involved highlighting the meaning of a single word (like part-of-speech tags). This is because we suspect that this is something the downstream model is likely to develop on its own from the rest of the training.

3.0.2 Final design

The annotation is made up of 4 components. These are reference-sets, concepts, propositions and the statement marker. For the rest of this document, the word "entity" will be used, which can mean any of those first 3. The annotation has a mostly tree-like structure (technically a graph), and an entity is effectively a node that other entities can point at, to build up meaning. The annotation looks akin to a JSON file or python dictionary. Python-style comments starting with a hashtag are also included.

```
1  A waterfall in the dessert.                          Input
2
3  \sep ──────────────────────────────────────────────────
4
5  Set1_1 = ("waterfall")
6  Set2_2 = ("in")                                      Reference Sets
7  Set3_3 = ("dessert")
8  ──────────────────────────────────────────────────────
9  Con1 = { # "in [the dessert]"
10   type: "Adposition",
11   relation: Set2,    # "in"
12   core: Set3         # "the dessert"                  Concepts/
13 }                                                     Propositions
14
15 Con2 = { # "A waterfall [in the dessert]"
16   type: "Modification",
17   core: Set1,         # "A waterfall"
18   modifiers: [Con1]  # "in the dessert"
19 }
20 ──────────────────────────────────────────────────────
21 Statements = [Con2]                                  Statement Marker
```

Figure 3.1: Minimal Annotation sample, see bellow for breakdown

3.0.2.1 Component breakdown

Reference-sets: Have the job of associating raw text spans from the input, to an entity that can be referenced by the other components. Every distinct part of the text that has meaning gets its own reference-set. This usually means each word, but there are 2 exceptions. First is things like full names (for example "John Smith"), which are captured by a single reference set, since the entire name refers to a single person. The second is when two different text spans reference the same entity. For example if in a sentence "Tom" and "he" refer to the same person, it would form a single reference set, $Set1 = (\text{“}Tom\text{”}, \text{“}he\text{”})$. All reference-sets are always defined at the start of an annotation, and should be defined in order of appearance in the input.

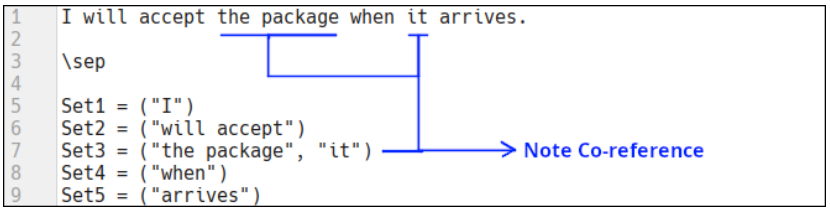


Figure 3.2: Example of Reference-Sets for a small sentence

Concepts: come in several types to handle different kinds of meanings.

- **Conjunction_and**: holds a list of entities that are part of an and-like statement (for example "cold and quiet" or things like lists).
- **Conjunction_or**: Same as above but for "or". Can optionally define whether it is inclusive/exclusive or ambiguous.
- **If**: Points to an antecedent and a consequent, forming an if/then statement.
- **Modification**: One of the two most common concepts. Holds a "core" entity and a list of modifier entities. The resulting concept refers to the meaning of the core entity, with the modifiers applied to it.
- **Adposition**: The other most common concept. Holds 2 entities, a "core" and a "relation". The concept refers to the core, introduced as an adposition using the relation. For example "under the blue house", would have "blue house" as the core, and "under" as the relation (this concept would likely then get attached to something else using a "Modification").

- **Ownership**: Refers to the ownership by someone/something. Used commonly for things like “my house”.
- **Negation**: Simply negates the input. Mostly used for negating verbs (“didn’t go”)

All concepts can use an “**anaphor_set**” field, which marks an entity as referring to the concept. This is for cases were a single word like “it” refers to an entire sub-sentence.
Propositions: Represent propositions found in the text. Can reference a subject, predicate, object, indirect_object, subject_complement and object_complement. They can additionally mark the tense, aspect and modality of the proposition, thought in practise this was only partially present in the final data (due to inconsistent annotation).

```
Give me that.

\sep

Set1_1 = ("Give")
Set2_2 = ("me")
Set3_3 = ("that")

Prop1 = { # Main proposition
    predicate: Set1, # "Give"
    object: Set3, # "that"
    indirect_object: Set2, # "me"
    type: "imperative"
}

Statements = [Prop1]
```

Figure 3.3: Example of a sentence which is just a proposition

Statement marker: Comes at the end and simply holds a list of top-level entities, each representing a “statement”. This is usually just 1 per sentence in the input.

3.0.2.2 Ambiguity and Edge cases

One challenge in NLP, is the ambiguity naturally found in language. For this project, the way to handle ambiguity was to maintain it if possible and pick any valid parse otherwise. For example the sentence “Tom and Ben received £5” could be interpreted as each person getting a separate payment, or both sharing the money together. For our representation, “Tom and Ben” would first get formed into one concept, which would then receive the payment. We consider this to maintain the ambiguity (as in, this annotation could be interpreted as both ways). In cases where this can’t be done, we simply want the annotator to pick any single valid interpretation of the sentence. Natural language can additionally have many “edge cases” which arguably require additional

rules/concepts to be added to the annotation schema. We choose to mostly ignore them when possible (assuming they are rare enough or can be easily filtered out) as we are mainly interested in targeting the 90% of "common speech".

3.0.2.3 Excluded Concepts

For practical reasons certain concepts were excluded from the data. This is mainly because including them would take too much additional time or effort. These are: questions, verbs found together with words like would/should/must, verbatim speech reports (anything said in quotes) and idiomatic expressions like "keep the team on their toes". Most of these where not worth the extra effort for this project.

3.0.3 Full examples

Bellow are two full examples to better demonstrate the annotation in use.

```
I will accept the package when it arrives.

\sep

Set1 = ("I")
Set2 = ("will accept")
Set3 = ("the package", "it")
Set4 = ("when")
Set5 = ("arrives")

Prop1 = { # Embedded proposition: "it arrives"
    subject: Set3,          # "it" (the package)
    predicate: Set5,        # "arrives"
}

Con1 = { # "when it arrives"
    type: "Adposition",
    relation: Set4,         # "when"
    core: Prop1             # "it arrives"
}

Con2 = { # "will accept [when it arrives]"
    type: "Modification",
    core: Set2,            # "will accept"
    modifiers: [Con1]       # "when it arrives"
}

Prop2 = { # Main proposition
    subject: Set1,          # "I"
    predicate: Con2,        # "will accept (when it arrives)"
    object: Set3,           # "the package"
    type: "declarative",
    tense: "future",
    aspect: "simple"
}

Statements = [Prop2]
```

Figure 3.4: Example 1

In example 1, note the use of a nested proposition.

```
The stars shone above, but they seemed distant and faded.

\sep

Set1 = ("stars")
Set2 = ("shone")
Set3 = ("above")
Set4 = ("but")
Set5 = ("they")
Set6 = ("seemed")
Set7 = ("distant")
Set8 = ("faded")

Con1 = {    # "distant and faded"
    type: "Conjunction_and"
    members: [Set7,Set8]
}

Prop1 = {    # Embedded proposition: "they seemed distant and faded"
    subject: Set5,    # "they"
    predicate: Set6,    # "seemed"
    object: Con1    # "distant and faded"
}

Con2 = {    # "but they seemed ..."
    type: "Adposition"
    relation: Set4,    # "but"
    core: Prop1    # "they seemed ..."
}

Con3 = {    # "shone [above] [but they seemed ...]"
    type: "Modification"
    core: Set2,    # "shone"
    modifiers: [Set3,Con2]    # "above" + "but they seemed ..."
}

Prop2 = {    # Main proposition
    type: "declarative"
    subject: Set1,    # "The stars"
    predicate: Con3    # "shone (above) (but they seemed ...)"
}


Statements = [Prop2]
```

Figure 3.5: Example 2 (taken from final synthetic dataset)

In example 2 note the mistake of “stars” and “they” not being part of a single reference set. Such mistakes is what we count as “minor errors” later on in the report, as the rest of the annotation is correct apart from this one logical error.

3.0.4 Observations/Takeaways from Data Design

When trying to design the data, looking at some existing annotations like universal dependences was helpful. Most of the help came from the various example sentences they use to demonstrate how the annotation works, as they can be used to “test” your annotation for any obvious blind-spots. Certain additional aspects of the annotation were dropped almost immediately after starting to produce samples as it became apparent they were redundant. Additionally, starting from real data and trying to describe it was significantly more productive that trying to design the schema in abstract.

Chapter 4

Data generation

The second stage of the project involved generating the dataset. First an existing LLM was iteratively fine-tuned to generate the data and then, once good enough, it was used to automatically generate the full dataset.

4.0.1 Fine-tuning phase

While a few models were considered at first, Qwen-3 4B quickly stood out as working the best, and was used for the rest of the project. Unsloth was used to perform Lora fine-tuning of the model, and for inference during this phase. The raw sentences to be annotated were generated by an LLM instead of using web-scraped text. This was done partially because it allowed to easily find self-contained full sentences, and also to control the variety of text. For each sentence, the LLM was given 2 random words to include, out of a list of the 3 thousand most common words on the internet. This gave a decent trade-off between variety and clean inputs. For the first few samples (around 80), the model did not have enough data to grasp the concept, so samples would have to be written fully by hand. Instead, Deepseek-R1 with an extensive prompt describing the annotation was used to "bootstrap" the dataset. Some other models were also tested but any non-reasoning model failed to follow the annotation schema. Even R1 struggled to keep a consistent schema, which made this first stage take a considerable amount of time. With around 80 samples the fine-tuned model was starting to follow the schema, but was still too inconsistent. By inspecting the outputs, some of the issues seemed to stem from the inconsistency present in the formatting of the data (for example what comments would say, indentation etc). A simple parser was written that would re-format samples to be consistent, and also catch some kinds of mistakes.

This noticably improved the performance of the fine-tuned model, making it faster to hand-correct its broken outputs to produce more data, rather than correct the mistakes from R1. The rest of the fine-tuning phase was mainly iteratively making data and re-training the model. In total, 674 "ground-truth" samples were created in 2 weeks.

4.0.2 Fine-tuning observations

Having a parser proved incredibly helpful in getting the process to work. This suggests that annotations which are mostly free-flowing text, might be harder to make reliably. Including comments not only made it easier to fix samples, but it also appeared to midly improve the reliability of the model, on the condition that the commenting style was consistent. Upgrading to a bigger model was tried, but seemed to offer no clear benefit. This is likely due to most failures coming from the model not understanding the annotation rather than its underlying language skills. While using a reasoning model to bootstrap the process helped, the inconsistencies in its outputs ended up costing a significant amount of time early on. In general, anything part of the formatting that can be standardized should be, as the model seemed to "latch on" to the differences and fail as a result. Empirically most errors seemed to stem from a few sources. Idiomatic or uncommon uses of speech often led to broken samples. Long input text was initially an guaranteed failure, but by the end the model could handle surprisingly long sentences. Interestingly, samples that were very close to an existing sample but slightly different, seemed to sometimes fail in very obvious ways. I suspect this was due to mild over-fitting when the data was small. Each "concept" in the annotation (for example adpositions) appeared to take 20-30 examples until the model would semi-reliably understand it. Asking a model to generate a few sentences with said concept and then annotating them together, proved to be an efficient way to make quick progress at the start. After around 200 samples, most failures seemed to stem from a single failure point (the model would make a single mistake, then the rest of the annotation would be thrown off). In retrospect, a script to quickly hand-correct the point of failure and resume generation, could make it significantly faster to fix samples, as it could eliminate the time taken to write the rest of the sample by hand.

4.0.3 Generation phase

For the full generation, the vLLM library was used for its performance. 500k raw sentences were generated, with checks to exclude certain contents, as mentioned before. An additional LLM based check was used to filter out idiomatic speech. The full generation took approximately 24 hours on an RTX A5000, costing around £5. The raw outputs where then passed through the parser, and failures removed, leaving 400k "clean" samples as the final dataset.

4.0.4 Data quality evaluation

From the final data, 100 random samples were tested to estimate the datasets quality. 62 were perfect. 16 got a pass, as they had no mistakes, but were an unusual valid interpretation of the text. 16 had small mistakes, defined as needing 1 "logical change" to fix, usually near the end. 6 were completely wrong. This means that approximately 78% of the data is correct with an additional 16% being almost correct. While usable, this was mildly disappointing, as we were originally aiming for around 85%+. It is also worth considering the effects of the filtering on the data distribution. Looking at the samples removed by the parser, they seem to use slightly more complicated speech on average. This implies that while excluding them from the data works, it also makes the dataset not as challenging/rich as it could be. Out of the samples with small mistakes, 6 involve the model not using the "negation" concept, which was also noticed during annotation. We believe this is manly due to it being less common in the input sentences, rather than an inherit limitation in the models ability. The model also seemed to only correctly annotate anaphora sometimes. Such examples were more common in the data, so the issues could stem from it being a harder concept to grasp. Lastly, co-reference was consistently annotated for certain uses but not for others. In cases where a name and a pronoun would refer to the same entity, the model would reliably get it correct. However, cases where "it" was used were a bit less reliable. We attribute this to a bias in the sentence generator, which produced such examples less often.

Chapter 5

Benchmark model training and Results

In order to test the effectiveness of the data, 2 small language models were trained, one using the tiny-stories dataset and one on a mixture between tiny-stories and our dataset. The models were then fine-tuned and evaluated on two different benchmarks.

5.1 Model architecture

A standard transformer architecture was used. They are 12-layers deep, with an embedding dimension of 256 and use standard flash-attention[5] with 4 heads. In the feed-forward blocks, an expansion factor of 4 is used, pre-activation LayerNorm[1], and GELU[9] activations. Rotary position embeddings[19] are used, and weight tying between the token embedding and final projection layer. A vocabulary size of 8k is used, mirroring the small vocabulary of the original tiny-stories models. Specifically the pre-trained vocabulary "englishcode-8000-consistent-v1" from the tokenmonster tokenizer, chosen for its claimed efficiency benefits. Lastly, drop-out of 0.2 is used in the output-projections of the feed-forward layers and in the attention operation.

5.2 Model pre-training

5.2.1 Data mixtures

To maintain a fair comparison, the models training data was token-matched. One was trained on the full tiny-stories dataset, while the mixed-dataset one had a random subset of samples removed, equal to the amount of tokens used by our dataset. This resulted in 974,778 tiny-stories samples being removed, in exchange for the new data.

5.2.2 Data Format and Loss masking

The synthetic annotation data was used with the comments included, since that appeared to improve performance during the fine-tuning phase. Due to time limitations a version without the comments wasn't tested.

There were additionally a few special changes with how the samples were trained on. The standard conversational input/output format found in most LLMs is not acquired during pre-training. Most models are simply trained on random equally-sized blocks of text sampled from the pre-training corpus. The conversational format is then acquired through special tokens being added, and a follow-up instruction-tuning phase. In this phase, the model is fed text separated into "user input" and "assistant output", with loss selectively applied only on the output segments. This teaches the model to respond to user queries, while not trying to directly predict or mimic the users input.

For this project, we wanted to use the conversational format directly during pre-training, to test a certain idea. Take as an example the sentence "John Smith was in Antarctica in 2000". If we were to simply train the model using standard next-word prediction on this sentence followed by its corresponding annotation, then it would be incentivised to try and memorise the "fact" that this fictional person was in Antarctica. However this setup is arguably flawed, since all we really want the model to learn is the logic associated with our annotation.

Now consider the same case, but the sentence is given inside a "user input" block, and the annotation in an "ai output" block, with the training loss only applied on the latter. At least in theory, the only information the model is required to learn in order to minimize its loss, is the logic required to generate the output given the input. In other words, it is not strictly required to memorize anything specific about the input sentence. We believe there could be a major advantage in using synthetic data this way, as there is evidence[7, 4] to suggest that a significant portion of parameters in existing LLMs is spent memorising various kinds of trivia. If this method of training allows us to avoid memorising unnecessary parts of the input, this could free up parameters to better model the logic we are trying to target.

The formatting used for our dataset samples was fairly basic, with the input asking for an annotation for a certain piece of text and the output being said annotation. However, to maintain a fair comparison, we needed to do the same for the tiny-stories dataset. This is because if only one model was trained with this format, it could potentially give it a small advantage in instruction following. For this reason, the

tiny-stories samples were all formatted with an input prompt asking for a small story. A few variations were used to avoid the model gaining a bias towards certain tokens always being present in the input. Lastly, a very basic instruction-style prompt was made, where on a 50% chance, the model would be asked to write a small story which includes certain words (akin to the instruction dataset in the tiny-stories paper). This was added to further reduce any potential biases from one model being better at following instructions simply due to being trained on 2 different tasks instead of one.

In terms of actual tokens, the input/output blocks were wrapped in 4 special tokens, **user_begin**, **user_end**, **ai_begin** and **ai_end**. Loss was only applied only to the tokens inside the "ai" block, and the "ai_end" token.

5.2.3 Optimizer and hyper-parameters

Both models were trained using the same setup. Sophia[14] was used as an optimizer, as it tends to regress faster and we have empirically found it to have slightly lower run-to-run variance. 1 full epoch was done over the data, with a batch size of 16. The learning rate started at 1e-3 and linearly dropped to 1e-4 by the end of training. The optimizer used b1=0.9, b2=0.95 and weight decay of 0.1. Standard cross-entropy loss was used. By the end of training both models had mostly plateaued.

5.3 Fine-tuning and Evaluation

Two different tests were used for evaluation. The "Blimp" benchmark was used as a broad test of language abilities. Perplexity and cross-entropy were measured on some out-of-domain data to see how the models perform when seeing new data.

5.3.1 Test 1: Blimp Benchmark

The Benchmark of Linguistic Minimal Pairs[23] or "Blimp" for short, is a benchmark which consists of 67 sub-datasets dedicated to various grammatical phenomena, with 1000 samples for each. Each sample consists of a good and bad sentence, and the model is tasked with identifying which is which. We took the first 500 samples of each category for our training set and the latter half as out test set. Fine-tuning hyper-parameters were tuned to simply minimize training loss. The training rate was 1e-4, and 2 epochs were done. The rest of the parameters were the same as the pre-training. Each sample was formatted as a 2-choice a/b question, asking the model to pick which sentence is most

correct. Over a 10 run average, the model trained only on tiny stories got a test accuracy of 74.46%, while the mixed-data model got 88.05%

5.3.1.1 Blimp per-category breakdown

One very useful property of the Blimp dataset is the way samples are segmented. Apart from the 67 sub-datasets each targeting a specific phenomena, samples are additionally grouped into 4 broad subfields of linguistics (morphology, syntax, syntax-semantics, semantics) or alternatively 12 "linguistics terms". This allows us to better look into how exactly the data affects the models performance.

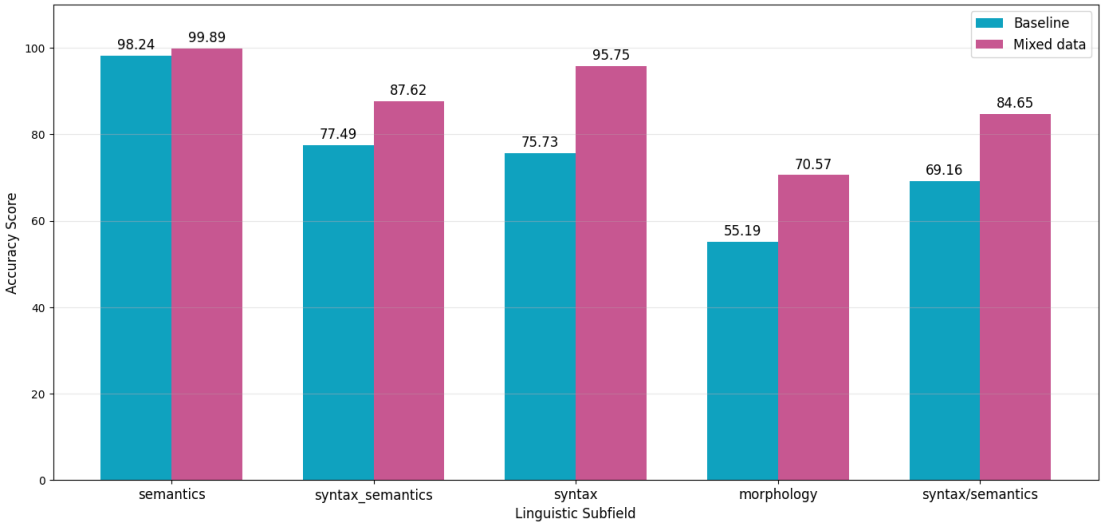


Figure 5.1: Blimp Benchmark by linguistic subfield

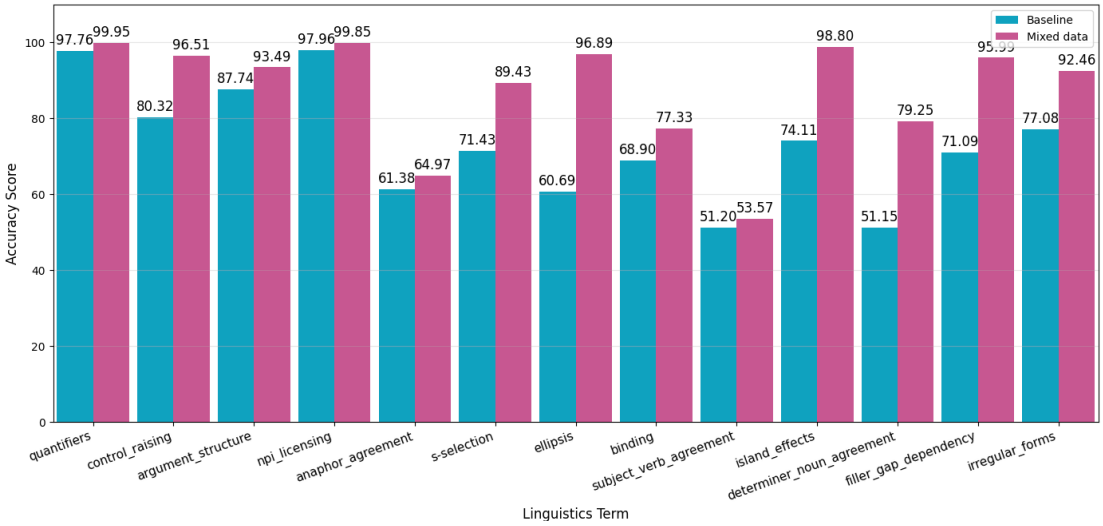


Figure 5.2: Blimp Benchmark by linguistic term

Four interesting observations stand out in these results. One is that the model with mixed data is a flat improvement across the board, rather than a trade-off. This is important, as it supports the idea that our data is actually more efficient to train on, rather that just a "different focus" for the model.

Second is that the improvement is much more pronounced in certain "linguistic terms" than others. This is likely because these were more relevant to the features our annotation tried to target.

Third is that morphology appears to have a decent improvement, even though our annotation did not directly target it. Our best guess is that the "morphology" tests involve using different tenses/forms of words to change the perceived syntax of a sentence. Even though our data does not directly annotate any morphological aspects of words, the influence these have on the syntax would be captured.

Lastly, there are certain subtasks where even the baseline model effectively saturates the benchmark, so there is little room for improvement. These are likely tasks where the necessary features needed to solve them have high overlap with the ones needed to auto-complete the text. (in other words, the standard training objective is already highly aligned with what we want, so datasets like ours aren't needed)

5.3.2 Test 2: Performance on out-of-domain data

While the two models were trained only on tiny-stories and our dataset, we further measured their perplexity on some other pieces of data. If our dataset provides a benefit, then in theory the model with mixed data should perform about the same as the baseline, or possibly better. Any "trivia" or novel knowledge should be equally new to both models. However the improved language abilities of the mixed-data model could allow it to generalize a bit better. We choose to measure on articles from Wikipedia-simple and math puzzles from Gsm8k.

Both of these datasets are significantly out-of-domain, compared to the original data. As such, the cross-entropy loss is quite high, stemming largely from the model being "surprised" by the unexpected format. This is mildly troublesome for our measurements, since to calculate perplexity we need to pass this loss through the exponential function which creates very high values (which could skew the average).

To help give a cleaner picture we instead take 4 similar measurements. First two are average perplexity and average cross-entropy loss. Second two are the same, but after we fine-tune the model on a tiny subset of the new data to have it "adapt" to the

new format. With these combined measurements we should have a clear view of which model is better at adapting to the new data.

For the pre-adaptation measurement on the wikipedia articles, we simply passed them to the model with the same prompt as if it were generating a story. For the post-adaptation the "input instruction" was a request to generate the article given its title. For the Gsm8k puzzles pre-adaptation we passed both the puzzle instructions and its step-by-step solution as the output to be predicted. For the post-adaptation, the input instruction was the puzzle definition and the output was the solution. For Wikipedia 6k articles were used in adaptation and for Gsm8k 800 puzzles. Fine-tuning parameters were the same as the ones used for Blimp, except with a smaller batch size of 8.

5.3.2.1 Measurement results

Model	**Dataset**	**Metrics**			
		Pre-adaptation		**Post-adaptation**	
		Cross-entropy	**Perplexity**	**Cross-entropy**	**Perplexity**
Baseline	Wikipedia	8.853	13809.8	5.827	449.4
	Gsm8k	9.654	28330.4	4.531	176.6
Mixed-data	Wikipedia	8.372	13090.9	5.757	415.1
	Gsm8k	9.314	20475.0	4.252	107.0

Table 5.1: Model Performance Results

On both datasets the model with mixed data shows a small but consistent improvement over the baseline. This further provides some support to the idea that the mixed data is a flat improvement to the model. Also notable is that the improvement also persists after the models have been fine-tuned to adapt to the new data. If the difference was only present pre-adaptation, it could suggest that the real difference is simply one model being more used to different data formats due to being trained on two broad tasks instead of just one.

Chapter 6

Conclusions

The two research questions we set out to answer were whether making datasets like this is feasible, and whether they are good enough to provide a benefit over regular data.

6.1 Data Generation

Designing and generating the data in a short time-frame seems feasible, but comes with some caveats. The final usable data was approximately 400k sentences, out of which approximately 78% were correct, with a further 16% having only minor errors. The computational cost was negligible. The two caveats are the time taken and the kind of data. First, the total time taken to reach the final dataset was approximately one and a half months. If this process is to be repeated many times, this is a workable period, though 1 month per dataset would be more preferable. We think this is possible since a decent amount of time could have been saved by knowing what to avoid. We also think that the final data quality could be higher in the same time-frame, as by the end of the hand-annotation period the rate of improvement was generally higher (since fixing samples became much easier). The second and more troublesome caveat is the kind of annotation being generated. It quickly became obvious that having a parser to fix and standardize samples significantly helped the process. If the desired data has a less structured formed than the one in our project, then having a system to catch broken examples could be significantly harder. We suspect that for such a project, some form of soft-validation, like using an LLM to double-check outputs could be useful.

6.2 Data Effectiveness

After testing, the data appears to be of high-enough quality to provide an improvement to the model, particularly in certain language abilities, without notably sacrificing performance in other areas. It is interesting to note that when looking at the per-category breakdown of the Blimp measurements, the improvement in some skills is quite significant, despite the data containing a decent amount of broken samples. This could imply that those features had very weak support in the baseline dataset, or that trying to learn them as a by-product is especially inefficient. The out-of-domain measurements also showed the mixed-data model having better transfer to new data, which would make sense if we assume that the benefit mostly comes from being better at understanding some basic language features. Overall, the main take-away is that data created in this way can be good enough to use for a down-stream improvement, which was one of the main points we wanted to show.

6.3 Self-Criticisms

We believe that this project provides some support to the idea that repeating a process like the one demonstrated to build up a large varied dataset could be a worthwhile investment. Making data in a small enough time-frame and of sufficient quality to make a difference seems possible. However there are a few things to consider.

6.3.1 Small scale Models

This test was done with relatively small models, under 50 million parameters. Before large scale models, a test in the 300 million parameter range would likely be better. While we expect similar datasets to benefit larger models, it is unclear how many would be needed to achieve a worthwhile difference when targeting larger models.

6.3.2 Data generation outlook

While data generation was successful, it's partially flawed when viewed in the context of the project. One of the main goals was to investigate the time and effort needed to produce one such dataset, to know if it's realistic to repeat the process many times. The slowest part of the project was the first half of the hand-annotation process where each new sample required significant time to prepare. This was mostly because we

expected the model to internalize the annotation schema by seeing enough examples. If instead we choose to fine-tune a model which could also see the schema definition as part of its input, or fine-tune a reasoning model, this could have required significantly fewer examples to reach a good result. We avoided this partially to avoid wasting compute resources, and partially to avoid unexpected issues which could arise with reasoning model fine-tuning. In a real setting however, both of those reasons would be scrutinized. The increased compute cost would likely still be minimal, and cheaper than the equivalent amount of work time. While reasoning models could give complications, those would likely be sorted out the first time the process was attempted, with every subsequent dataset being streamlined. In essence, the criticism would be that we approached the data generation with the mindset of producing a single dataset under limited resources, even though it was meant to be a proof-of-concept for a real setting which arguably would value the cost of labour more than computational resources. The silver lining is that our method is an over-estimate of the time needed.

6.3.3 Beyond core abilities

The project started out with the broad idea of synthetic datasets for "semantic understanding of text" by forcing the model to explicitly "write out" implicit logic found in the input, in the form of an annotation. While this worked, the dataset we produced was rather structured and syntax-focused. Targeting syntax as our "implicit logic" raises the question of how many times this process could realistically be repeated. A fair criticism would be that after some core abilities like syntax or morphology are worked on, any remaining abilities would be too specific to warrant the effort of a separate dataset being produced. This is partially countered by the argument, that even if only a handful of skills get a significant boost, then that is a worthwhile investment, but we don't believe our project alone provides sufficient evidence of how much of a jump in performance could be achieved, as we focused on smaller models. Instead, we suspect tackling the idea of rarer skills being too specific to warrant their own dataset is where further progress could be made. We touch upon this again bellow.

6.4 Overall Conclusion

There are three key points from this project we see as important.

First is that the process of creating one such dataset seems viable, and fairly reliable. Most of the slow-downs in the project were mostly due to not having the setup streamlined. There were no points where significant issues arose, and the only thing we would consider a task-specific "hack" was the implementation of a parser, which was fairly basic and mostly amounted to standardising formatting and catching really basic errors. As such, we think it's fair to suggest that this method could be scaled up without much issue, to build a corpus of small datasets, which would likely be enough for a real-world improvement. We do however note that datasets with more fluid/unstructured outputs might be different enough where a slightly different method might be better. This answers our two original research questions.

Second is the perspective of this process acting as a form of knowledge distillation. Most of the manual effort was simply creating enough data to tune our pre-trained model to generate the dataset. However this was purely alignment, as we are relying on the underlying model's ability to already understand the logic we are trying to extract. The resulting dataset can be seen as a "distillation" of Qwens ability to understand the structure of a sentence, which we then passed onto a smaller model. Perhaps thinking of it in this way could be a good source of ideas on how to further improve it.

Third is that the manual work undertaken seems like it could be a target for further automation. Having done the process by hand, we believe it could plausibly be decomposed into a few smaller subtasks, each of which could then be done by other models. For example, take the step of designing the annotation. Currently, we had to manually choose what features to target and how. Instead, a model could be trained to take in an input and respond with a bullet-point style list of various "patterns" it thinks are present/interesting (for example co-reference etc). This could then be done for a large corpus, and clustering could be used to find common patterns found in many samples. A separate model could then be asked to try and engineer an annotation etc. While fully automating the process would likely be too unreliable, it seems like a bit of automation could be used as a lever to speed up the process even further.

6.5 Future Work

A lot of work could be done fine-tuning various parts of the process, but it would likely be task-specific. Instead, here are some broader directions we think this research could move in.

- **Automate Feature selection:** If possible, having a system that can digest a large corpus of raw data and "highlight" various kinds of features we could build datasets for, could potentially be very valuable. It is unclear how feasible this is, but we expect a rudimentary version should at least be viable.
- **Automate Annotation design:** As above, the annotation design could likely be sped up using existing models. We attempted using some existing models (R1, Claude 4) to try and improve the schema at the start of the project. While they weren't very helpful when asked to improve it directly, they could reliably find issues with the design. As such, we think some work could be done in this area.
- **Combine with self-consistency techniques:** There is a decent amount of work in getting models to be more self-consistent. This usually involves some kind of process where the model answers the same question in different ways and is then trained on the "most common" answer. Some similar system could be used to further reduce the amount of hand-annotated samples needed to tune a model to generate a dataset.
- **Investigate softer annotations:** Our final dataset had a fairly rigid annotation schema. We think that softer annotations might pose some new challenges (we suspect some sort of LLM verification might be needed), so it could be worthwhile to perform a similar experiment in that area.
- **Dynamic annotations:** Not all pieces of data contain the same underlying logic. It would be interesting to investigate the use of "dynamic" annotations, where the definition is slightly specific to the kind of sample being annotated.
- **Bigger Experiment:** While a good first step, this project performed testing on a fairly small scale, mostly due to practical limitations. An obvious next step would be to scale up to a few datasets, and around hundred-million parameter models to start measuring some real-world performance.

Chapter 7

Closing notes

All core parts of the project are now covered, so there are a few last notes to mention.

7.0.1 Further Particulars in appendix

There are many notes about the project that are important to include to justify or explain various design choices, but are too verbose to include into the main text. While I consider them "non-optional" for the full report, they are too specific to put into the main body. They are included in appendix A, in a slightly less structured Q&A format.

7.0.2 Data & Code

The final dataset and the code for the project are included in the supporting materials. The code is complete but in a semi-runnable state as it was mostly a series of scripts being modified as needed.

7.0.3 Personal outlook

In general I view the project as a success, though certain areas near the end ended up being less complete than I would like (for example was aiming for 3-4 benchmarks but couldn't fit them due to schedule). I was expecting the idea to work going in, but was still nice to see the benchmark results actually show an improvement at the end. The main two challenges with the project were the amount of pieces needed, and the switching between active/creative work and monotonic hand-annotation.

Overall pleased.

Bibliography

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Daixuan Cheng, Yuxian Gu, Shaohan Huang, Junyu Bi, Minlie Huang, and Furu Wei. Instruction pre-training: Language models are supervised multitask learners. *arXiv preprint arXiv:2406.14491*, 2024.

[3] Cheng-Han Chiang and Hung yi Lee. On the transferability of pre-trained language models: A study from artificial datasets. *ArXiv*, abs/2109.03537, 2021.

[4] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers. *arXiv preprint arXiv:2104.08696*, 2021.

[5] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

[6] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[7] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*, 2020.

[8] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[9] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[10] Michael Y Hu, Jackson Petty, Chuan Shi, William Merrill, and Tal Linzen. Between circuits and chomsky: Pre-pretraining on formal languages imparts linguistic biases. *arXiv preprint arXiv:2502.19249*, 2025.

[11] Ran Iwamoto, Issei Yoshida, Hiroshi Kanayama, Takuya Ohko, and Masayasu Muraoka. Incorporating syntactic knowledge into pre-trained language model using optimization for overcoming catastrophic forgetting. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10981–10993, Singapore, December 2023. Association for Computational Linguistics.

[12] Mojan Javaheripi, Sébastien Bubeck, Marah Abdin, Jyoti Aneja, Sebastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, et al. Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 1(3):3, 2023.

[13] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023.

[14] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training. *arXiv preprint arXiv:2305.14342*, 2023.

[15] Pratyush Maini, Skyler Seto, He Bai, David Grangier, Yizhe Zhang, and Navdeep Jaitly. Rephrasing the web: A recipe for compute and data-efficient language modeling. *arXiv preprint arXiv:2401.16380*, 2024.

[16] Isabel Papadimitriou and Dan Jurafsky. Injecting structural hints: Using language models to study inductive biases in language learning. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8402–8413, Singapore, December 2023. Association for Computational Linguistics.

[17] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023.

[18] Corby Rosset, Chenyan Xiong, Minh Phan, Xia Song, Paul Bennett, and Saurabh Tiwary. Knowledge-aware language model pretraining. *arXiv preprint arXiv:2007.00655*, 2020.

[19] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[20] Yuanhe Tian, Yan Song, and Fei Xia. Improving relation extraction through syntax-induced pre-training with dependency masking. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1875–1886, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[21] Georgios Tziafas, Konstantinos Kogkalidis, Gijs Wijnholds, and Michael Moortgat. Improving BERT pretraining with syntactic supervision. In Ellen Breitholtz, Shalom Lappin, Sharid Loaiciga, Nikolai Ilinykh, and Simon Dobnik, editors, *Proceedings of the 2023 CLASP Conference on Learning with Small Data (LSD)*, pages 176–184, Gothenburg, Sweden, September 2023. Association for Computational Linguistics.

[22] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[23] Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R Bowman. Blimp: The benchmark of linguistic minimal pairs for english. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020.

[24] Zhixian Yang and Xiaojun Wan. Dependency-based mixture language models. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7758–7773, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[25] Zhuosheng Zhang, Yuwei Wu, Hai Zhao, Zuchao Li, Shuailiang Zhang, Xi Zhou, and Xiang Zhou. Semantics-aware bert for language understanding. In *Pro-*

ceedings of the AAAI conference on artificial intelligence, volume 34, pages 9628–9635, 2020.

Appendix A

First appendix

The following section covers some additional details readers may be interested in regarding the project. It is a mostly un-ordered list of things that were too specific to fit into their respective sections, but could still be important.

A.0.1 On Data-generation phase

A.0.1.1 Why not fine-tune a reasoning model?

We considered fine-tuning a reasoning model to generate the annotation. Part of the reason we chose not to, is because of the increased complexity. We suspected that the underlying task is not actually that hard, and most of the fine-tuning is essentially just teaching the model to "follow the schema". This is also the reason why the fine-tuned model was never given the "definition" of the annotation as an input, and instead was just expected to memorise the schema. Additionally, not using a reasoning model reduces inference cost, and allows us to make more data. If this project was to be repeated, we suspect a better choice would be a 3-stage approach. Initially fine-tune a reasoning model during the hand-annotation phase. Once it gets good enough to produce samples reliably, produce a slightly larger dataset and train a "standard" model. Then use that to efficiently generate the full data.

A.0.1.2 Would a real model not need the definition?

In our experiments the model was simply told to "annotate the input". This obviously would not work in a real setting, as there are multiple ways one could annotate something. However, we think that even in a real setting, giving the entire annotation schema as part of the input instruction might be a mistake. You could instead simply give the annotation

a code-name like “syntax_1”, and expect the model to learn it. While expecting the model to memorise the schema + a random name may seem weird, it is likely a drop in the bucket compared to the amount of other trivia the model is also memorising during pre-training. Additionally, it removes the cost of having the entire definition in the input, allowing for more training.

A.0.1.3 How often was the model re-trained during data annotation?

Initially every 5-6 samples, as it took almost no time, and we wanted to get it to improve as quickly as possible. As the dataset grew it became closer to 20-30 as each sample had less of an effect. By the end it was basically once a day, or around every 50 samples.

A.0.1.4 What does the parser check for?

A few checks are done to spot “broken” samples. Each concept/proposition must only use the acceptable fields for its type. Sets/Concepts/Propositions need to be defined in order, with no gaps in the ids. (So defining Concept 1, then Concept 3 would be invalid, since 2 is missing). Entities can’t be referenced without being defined. Lastly there is a “floating node” check, which basically checks that every entity that is defined needs to be somehow linked to the final statement marker, directly or indirectly. That last check is the one that fails most often, as it is a common side-effect of the model getting confused.

A.0.1.5 Why not fine-tune on samples that pass the parser?

We considered doing fine-tuning by selecting a bunch of samples that pass the parser checks, and add them to the dataset without manual inspection. We decided against, as failing the parser check seemed to be a decent check for when the model has largely miss-understood a sample. If we trained on this signal, it could produce a situation where the model learns to “fail in a way that fools the parser”, so we decided to avoid it. (i.e. Goodhart’s law)

A.0.2 On evaluation phase

A.0.2.1 Why those benchmarks?

We were originally aiming to use winogrande, hellaswag and a few others, but they proved too difficult for models of this size. When fine-tuned on them, models effectively

defaulted to always just picking 1 of the available options regardless of the input (which effectively means that they are so weak that the fine-tuning signal from trying to understand the benchmark is effectively just noise). In retrospect this should have been expected from the start, but we thought they would at least be able to score a minimum score above random guessing.

A.0.2.2 Why fine-tune on benchmarks?

We are effectively trying to measure if the internal feature-engineering of the models is better as a result of the different pre-training data. Fine-tuning on such small datasets doesn't really teach the models anything new, besides just tuning them to "understand" which feature to express. As such, fine-tuning on the benchmarks allows us to eliminate any uncertainty of one model being better simply because they "happen" to better understand the prompt.

A.0.2.3 Could the uncommon optimizer/tokenizer bias the results?

The tokenizer and optimizer used are a bit uncommon. This was mostly done since we wanted to push the models to be as good as possible, to hopefully get a cleaner "measurement" of the performance difference. Since both models used the same setup, we don't think it biases the result in any way. The one thing which could be of note is that the tokenizer is supposed to be tuned in a way that performs a bit better for code/math (by more avoiding grouping relevant tokens such as numbers together). It is plausible that a model with an older tokenizer which doesn't do this, might struggle more to output the "structured" style of the annotation, but as far as we are aware, most bleeding edge models are trying to have tokenizers with similar characteristics.

A.0.2.4 Why not math-problem style questions?

When originally planning out the project we considered text like gsm8k could be a good style of text to "annotate" but ended up avoiding it as it seemed like normal text was easier to get a working project with, and arguably easier to measure.

A.0.2.5 Could you better measure the effect on memorisation?

Initially, the detail about training only on the output of the annotation as a way to avoid memorisation was meant to have a bit more focus. However it became evident that trying to measure it exactly was a bit tricky. It would also arguably be irrelevant if we

couldn't show that the general process gives some improvement in the first place. (after all, the effect on memorisation is really just a possible argument for why this could work). We had some ideas to use some synthetic "trivia sentences" to show that the masking really does avoid picking up unrelated facts into the model, but was cut to focus on the main part of the project.

A.0.2.6 Isn't the un-even sample size a major issue?

The reason why models are trained on equal-sized random blocks of text is performance, as it allows to better utilize the hardware. Pre-training using input/output style samples is indeed non-ideal as it required us to effectively waste some performance on "padding" tokens. However in practice, a much more efficient packing could be done, we just didn't implement it for simplicities sake. Additionally, for really small models, the small amount of compute inefficiency still present, is arguably acceptable, as the main focus might be squeezing more performance into a model with a smaller footprint, rather than training cost.